

[START HERE / ESTIMATES](#)

A small project that meets the manual

Build a guardian-controlled vaccination-sharing demonstration with **three Solidity contracts, one Python console app and local JSON**. Deploy and test on the local Hardhat network from your labs. No website, public testnet, Java backend or Fabric-X infrastructure is needed for this version.

This plan supersedes the earlier 54-class Java architecture and its developer PDF. It is a revised implementation specification, not a report that the new software already exists. Existing source files have not been rewritten by this PDF task.

Developer	Responsibility	Core work	Plan with buffer
1	IdentityRegistry + ConsentManager	10-14 h	12-16 h
2	Reward token + Solidity unit tests	8-12 h	10-14 h
3	Python app, local JSON and disclosure	8-12 h	10-14 h
4	Hardhat, RPC integration and measurements	10-14 h	12-16 h
5	Research, report, UML and demo verification	8-12 h	10-14 h

What the estimates mean

Core effort totals **44-64 person-hours**. A two-hour buffer per person gives **54-74 person-hours** for the team. Across seven days, the individual plan averages about 1.5-2.3 focused hours per day. These are planning estimates, not guarantees or time already spent.

Assumptions: you can reuse the lab setup, have basic Solidity/Python familiarity, and complete the Day 1 connection check. Learning both tools from scratch or resolving unknown course requirements can exceed these ranges. Keep the buffer for integration, not extra features.

The demonstration

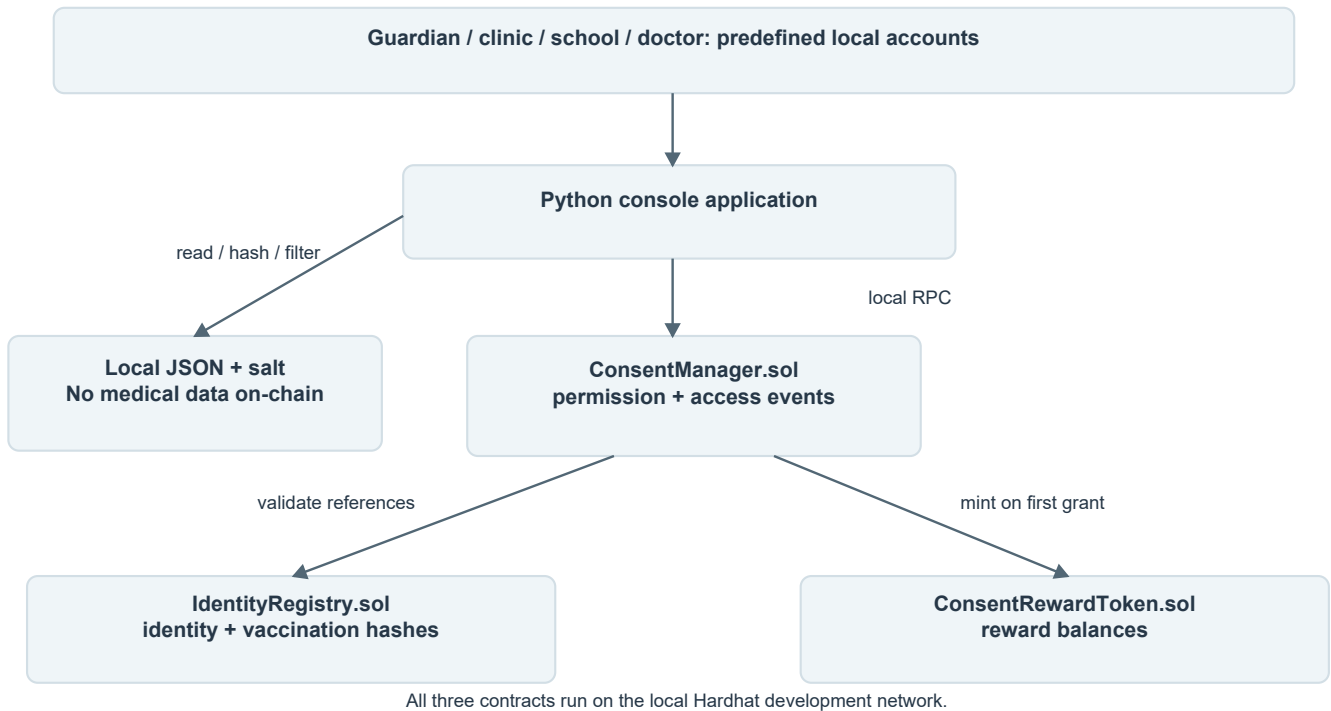
Register synthetic identities; attest one local vaccination record; deny school access; grant consent and reward the guardian; show school status and doctor details; revoke or expire consent; detect a changed JSON file; display the access audit and gas results.

Reading order

Pages 2-5 explain architecture, data and contract behavior. Pages 6-10 are individual assignments. Pages 11-13 define the week schedule, tests, measurements and submission checklist.

DESIGN / LOCAL FIRST

Architecture and roles



Actor	Small prototype responsibility
Guardian	Registers identity; represents one child; grants/revokes school or doctor scope.
Child	Subject of the local record, not an independently authenticated account.
Clinic	One preapproved signer, fixed at deployment, registers a vaccination hash for the guardian.
School / doctor	Separate registered accounts requesting different scopes. Neither gets access merely because of its label.
Deployer	Deploys contracts, fixes the trusted clinic and wires the reward minter once; cannot grant another guardian's consent.

Boundaries

The Python app is the trusted disclosure point. It reads the guardian's local file, computes its commitment, sends an access transaction and releases only the approved fields. The chain stores identity/record hashes, consent metadata and events - never the medical JSON or its salt.

Use local lab accounts selected through the console. This is an explicit demo identity mechanism, not production authentication. A person with direct filesystem access can copy plaintext outside the app; the blockchain cannot prevent that or retract information already received.

The UML above is a component view. ConsentManager depends on the registry for identity/evidence and on the token contract for rewards. Python talks to all three through one RPC helper.

DESIGN / DATA OWNERSHIP

Local JSON and minimal data model

Attribute	On-chain?	Off-chain?	Hashed?
Synthetic ID / email	Hash only	Local identity fixture	Yes, salted
Child ID and vaccination values	Record commitment only	Local vaccination JSON	Yes, as file bytes
Guardian and requester addresses	Yes	Demo account labels	No
Scope, expiry and revoked flag	Yes	Displayed by Python	No
Private commitment salts	No	Separate local metadata	Included in hash input
Access decision, scope and time	Event metadata	Displayed from receipts/logs	No
Reward balance	Yes	Displayed by Python	No

One fixed vaccination card

Use one synthetic child and a single MMR event. Save the JSON once before clinic attestation. Runtime editing, new doses, version histories and guardian transfer are outside this week's scope. The tamper demonstration modifies a copy of the file and restores the original afterward.

```
{
  "child_id": "child-demo-001",
  "vaccinations": [{
    "vaccine": "MMR",
    "covers": ["measles", "mumps", "rubella"],
    "date": "2026-03-12",
    "clinic": "Clinic A (demo)",
    "batch": "ABC123-DEMO"
  }]
}
```

Simple, explicit hash rule

Vaccination commitment = SHA-256(prefix || salt32 || exact JSON file bytes), with prefix VACCINATION:v1 followed by one newline and a fresh 32-byte salt. Identity registration uses the same format with IDENTITY:v1, a separate salt and the identity fixture bytes. Store salts locally as base64; send only bytes32 commitments to the registry.

No canonicalization is needed for this frozen fixture. Whitespace changes count as changes. Python must hash and parse the same bytes; do not reread another file after authorization. Reject malformed JSON or a missing salt. Use a configured data directory and fixed demo filenames, never an arbitrary path supplied by a requester.

School output: verified status only. Doctor output: vaccine and date only. Missing or invalid evidence is UNAVAILABLE/INVALID, not a clinical NO. Do not claim the absence of a record proves a person is unvaccinated.

DESIGN / THREE CONTRACTS

Contract interfaces and consent rules

These are proposed public interfaces to freeze on Day 1, not existing implementations. Use bytes32 hashes, account addresses and two numeric scope codes: 1 = MEASLES_STATUS, 2 = VACCINATION_SCHEDULE.

Contract / operation	Expected behavior
IdentityRegistry.registerUser(identityHash)	Registers msg.sender once. Reject zero hash; expose only hashes/references.
IdentityRegistry.registerVaccination(guardian, recordHash)	Only the fixed trusted clinic can attest. Guardian must be registered. One nonzero commitment per guardian; no replacement in the demo.
IdentityRegistry.getUserInfo(account)	Returns registered flag, identity hash and vaccination hash; no raw identity or health data.
ConsentManager.grantConsent(requester, scope, durationDays)	Owner is msg.sender. Require registered parties, supported scope and duration 1-365 days. Reject an already-active duplicate.
ConsentManager.revokeConsent(requester, scope)	Only changes the caller's own grant. Repeating revocation is safe; never modifies another owner's consent.
ConsentManager.checkAccess(owner, requester, scope)	Read-only helper returning current permission and reason; does not itself log a data access.
ConsentManager.requestAccess(owner, scope, observedHash)	Requester is msg.sender. Check registration, evidence, consent and submitted commitment; emit a granted/denied attempt and return its outcome.
ConsentRewardToken.balanceOf(account)	Returns non-transferable reward units. Balance is never consulted to authorize data access.

Consent and reward state

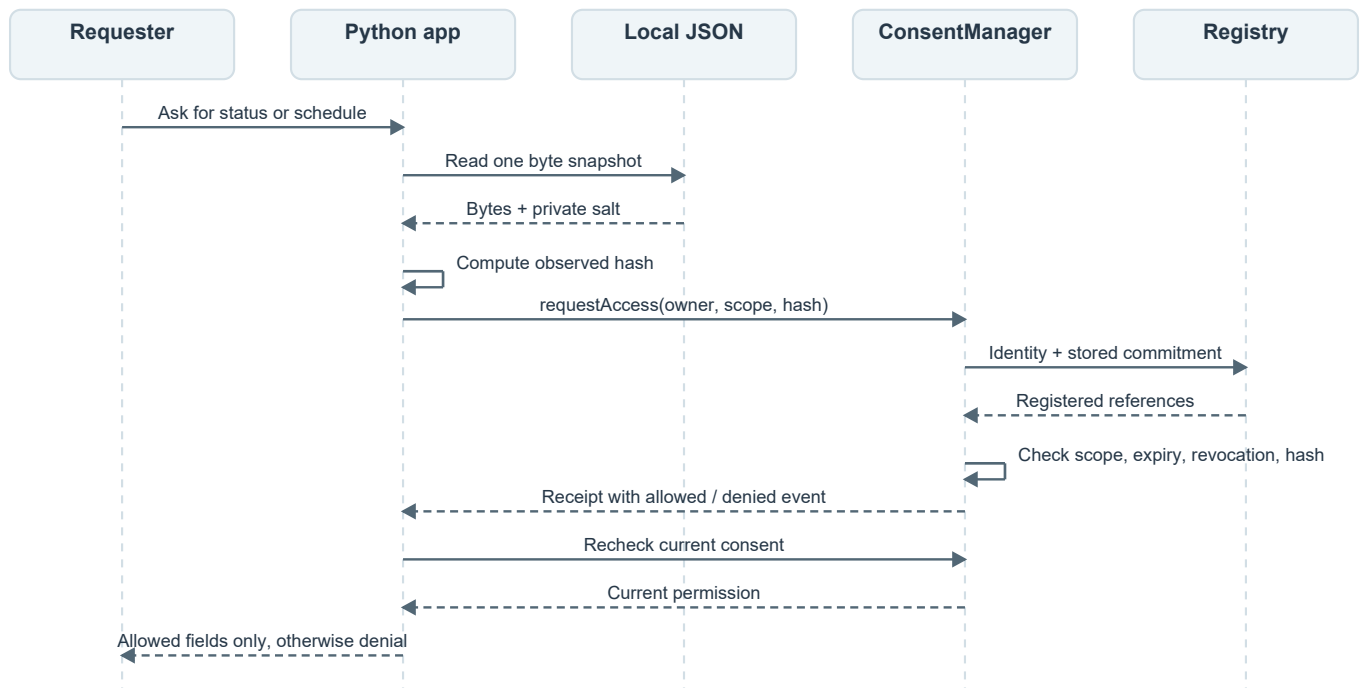
Key grants by (owner, requester, scope). Store expiry and revoked state. Validity requires current time strictly less than expiry. Convert durationDays once using Solidity days; simulate time advancement in tests rather than lowering the minimum duration.

Award one reward unit only on the first successful grant for that exact tuple. Remember the reward permanently across expiry/revocation/regrant so toggling consent cannot farm rewards. Regrant after expiry/revocation is allowed but yields no second reward for the tuple. This is not a Sybil-resistant economic system; use predefined demo identities.

Token setup: the deployer sets a single ConsentManager minter once. Only that minter can call mintReward(owner). Track balance and total supply; no transfer/trading API is needed. If your lab mandates a token standard, reuse that standard's lab implementation rather than adding new economics.

DESIGN / SEQUENCE UML

Access workflow and audit semantics



A denied request commits an event; it must not revert after emitting that event.

Events to implement

IdentityRegistry: UserRegistered and VaccinationRegistered. ConsentManager: ConsentGranted, ConsentRevoked and AccessAttempt. ConsentRewardToken: RewardMinted. An access event identifies owner, authenticated requester, scope code, block timestamp, allowed flag and a compact reason code. Do not put vaccine names, dates, batch, salts or JSON in events.

Well-formed denied access requests must succeed as transactions with an allowed=false event. Reverting after emitting would erase the event. Invalid signatures, malformed transactions or RPC failure cannot create an on-chain log: report those as application/network failures and release nothing.

Python release rule

Read one local byte snapshot, validate it and calculate the observed commitment. On a missing/invalid local record, submit the reserved zero observed hash so the well-formed request can be logged as denied. Submit requestAccess from the selected requester and wait for a successful receipt. Read the matching contract event; a transaction hash alone is not authorization.

Release data only if the event allows it, the snapshot matches the registered commitment and the current consent recheck still passes. No tokens move during access. If the final recheck fails, submit another denied access attempt when the network is available and release nothing.

Be precise when presenting

The contract checks a submitted hash; it cannot prove the caller actually read a file. The event records an authorization attempt, not physical delivery. Python enforces projection using the checked bytes. Revocation committed before the last check blocks release; already-shared or in-flight data cannot be recalled. Do not claim stronger guarantees.

DEVELOPER 1 / 12-16 HOURS INCLUDING BUFFER

Identity and consent contracts

Ownership: contracts/IdentityRegistry.sol; contracts/ConsentManager.sol. Pair with Developer 2 on their matching Solidity tests.

Work block	Core estimate
Freeze state, signatures and events	1-2 h
Implement registry and trusted-clinic checks	2-3 h
Implement consent, access logging and rewards hook	4-5 h
Review, tests and integration fixes	3-4 h

Implement in this order

1. Implement wallet registration using msg.sender and a nonzero identity commitment. Prevent duplicate registration; never accept a requester address as proof of caller identity.
2. Fix one trusted clinic in the registry constructor. Permit that clinic to register one frozen vaccination commitment for a registered guardian. Reject arbitrary callers, missing owners and replacement hashes.
3. Implement grants keyed by owner/requester/scope with duration 1-365 days. Check exact requester and scope; deny at expiry. Own-grant revocation must not affect another guardian.
4. Implement requestAccess as a state-changing audit operation. Every well-formed denied request emits its denied reason and returns normally. Check the supplied observed commitment against registry evidence.
5. Implement a permanent rewarded-tuple flag and call the restricted reward minter on the first eligible grant only. A failed mint must not leave an apparently successful grant; keep the transaction atomic.
6. Agree event layouts and reason codes with Developer 4 before coding the Python event reader. Review Solidity tests with Developer 2 and fix the logic they expose.

Handoff

Day 1: signatures/state/events to Developers 2 and 4. Day 3: compilable contracts and positive/negative calls. Developer 2 owns the test files but you remain responsible for fixing these contracts.

Done when

- Another account cannot attest a vaccination or alter someone else's grant.
- Wrong requester/scope, unregistered users, expired/revoked consent and hash mismatch are denied and logged.
- Repeated active grant is rejected; regrant after revocation works without duplicate reward.
- Read methods expose only registered references; every mutation has an appropriate event.

Time boundary: 10-14 hours core work + 2 hours integration contingency = 12-16 hours planned. Escalate blockers early; no bonus features within the buffer.

DEVELOPER 2 / 10-14 HOURS INCLUDING BUFFER

Reward token and Solidity unit tests

Ownership: contracts/ConsentRewardToken.sol; test/IdentityRegistry.t.sol; test/ConsentManager.t.sol; test/ConsentRewardToken.t.sol. Use the lab's test-folder convention if different.

Work block	Core estimate
Build restricted reward contract	2-3 h
Write registry and consent test cases with Dev 1	3-4 h
Write reward tests and test-result evidence	2-3 h
Cross-review and fix test failures	1-2 h

Implement in this order

1. Implement balances, total supply and a RewardMinted event. Allow deployer-controlled one-time minter setup and reject unauthorized minting. Keep reward units non-transferable unless the lab explicitly requires another standard.
2. Configure tests as Solidity .t.sol files run by Hardhat, following Lab 3. Do not replace the required Solidity unit tests with only Python or TypeScript tests.
3. Cover identity registration, duplicate/zero values and clinic-only frozen attestation. Cover grant duration boundaries, ownership, exact scope/requester and exact expiry.
4. Assert the AccessAttempt event is present for denied well-formed calls; do not merely assert that something reverted. Check that no token transfer or mint occurs during access.
5. Test first grant reward, duplicate active grant, revoke-and-regrant, expiry-and-regrant and a different requester/scope. The same tuple receives only one lifetime reward.
6. Give each test a stable ID and one sentence explaining why its functionality matters. Export actual pass/fail evidence for Developer 5; do not prefill success or gas values.

Handoff

Agree the test harness and contract interfaces on Day 1 with Developers 1 and 4. Contract owners resolve logic bugs; you own reproducible unit-test coverage and results.

Done when

- All three correctly named Solidity suites compile and run in the lab Hardhat environment.
- Tests cover both permitted and denied behavior, not only happy paths.
- Only the configured ConsentManager can mint; repeated reward attempts cannot inflate balances for the same tuple.
- The test report ties every tested behavior to a requirement and observed result.

Time boundary: 8-12 hours core work + 2 hours integration contingency = 10-14 hours planned. Escalate blockers early; no bonus features within the buffer.

DEVELOPER 3 / 10-14 HOURS INCLUDING BUFFER

Python console, JSON and minimal disclosure

Ownership: app/main.py; app/records.py; app/disclosure.py; data/examples/vaccination_record.json; local identity/salt fixtures. Developer 4 owns app/chain.py.

Work block	Core estimate
Create and validate synthetic JSON + salts	1-2 h
Implement save/read/hash helpers	2-3 h
Build console actions and scoped responses	3-4 h
Test tampering, denial and handoffs	2-3 h

Implement in this order

1. Create the one-child/one-vaccination fixture and identity fixtures using synthetic values only. Save locally under a configured data root; generate separate 32-byte salts and keep them out of ledger payloads.
2. Read exact bytes once, validate the expected JSON fields and compute the agreed SHA-256 commitment. Use that same snapshot for later display. Fail safely on malformed/missing data or salt.
3. Build a plain terminal menu: select demo actor, register, clinic-attest, grant, revoke, school-check, doctor-view, rewards and audit. Predefined local accounts replace a login/registration website.
4. Call Developer 4's RPC helper for all transactions. Show pending, denied and unavailable separately. Read the committed access event and perform the final permission check before showing any health data.
5. School output contains only authorized verified status. Doctor output contains vaccine/date only. Never return the file path, raw JSON, batch or salt. Missing evidence is unknown/unavailable rather than a medical NO.
6. Add a repeatable tamper demonstration using a copied fixture and a restore step. Do not build general medical-record editing, encryption, databases, version recovery or a GUI.

Handoff

Day 1: agree request/result shapes with Developer 4. By Day 3: file/hash functions tested against fixed bytes. Day 4: consume real receipts/events rather than mock permission responses.

Done when

- The record is physically saved locally and loaded without a cloud service.
- Hashing is reproducible; altering any file byte breaks the original commitment.
- No health values appear on denied/unavailable responses or ordinary logs.
- One menu can run the complete school and doctor demonstration with no source edits.

Time boundary: 8-12 hours core work + 2 hours integration contingency = 10-14 hours planned. Escalate blockers early; no bonus features within the buffer.

DEVELOPER 4 / 12-16 HOURS INCLUDING BUFFER

Hardhat setup, integration and measurements

Ownership: Lab-derived Hardhat config/deployment scripts; app/chain.py; integration/demo_workflow.py; evaluation/measure.py; generated deployment addresses/ABIs and result tables.

Work block	Core estimate
Start lab environment and freeze interfaces	2-3 h
Deploy/wire contracts and implement RPC helper	3-4 h
Run full workflow and repair integration	3-4 h
Measure gas/timing and document commands	2-3 h

Implement in this order

1. Reuse the lab Hardhat version and Solidity-test setup. Before other work depends on it, compile a tiny lab contract, deploy locally and call it from Python. Pin the working dependencies and record commands.
2. Deploy registry and reward token, deploy ConsentManager with their addresses, then set the token's minter once. Verify addresses and trusted clinic. Export ABI/address configuration; never hardcode real private keys.
3. Implement Python RPC operations with the selected local account, successful receipts and decoding of the exact AccessAttempt event. A missing/reverted receipt or missing event is unavailable, never permission.
4. Create a repeatable integration workflow covering registration, frozen clinic attestation, first grant reward, permitted read, revocation/expiry denial and hash mismatch. Coordinate the final permission recheck with Developer 3.
5. Measure actual deployment gas for each contract and gasUsed for core functions. Separate first rewarded grants from later unrewarded grants and granted from denied access. Report sample counts and averages.
6. Run small local scenarios with 1, 5 and 10 requesters. Time send-to-receipt and send-to-event-decoding using a monotonic clock. Record local automining and machine settings; these are demo measurements, not public-chain throughput claims.

Handoff

The Day 1 Python-to-Hardhat call is the main schedule gate. Give Developer 3 a stable helper interface and Developer 5 exact commands, deployment receipts and measured CSV tables.

Done when

- A clean checkout follows documented commands to compile, deploy, run Solidity tests and start the Python demo.
- All three contracts are connected to the same local deployment; unauthorized minter setup fails.
- The integration workflow uses real transactions, not a simulated permission dictionary.
- Evaluation tables are calculated from receipts and timings, with units and scenario labels.

Time boundary: 10-14 hours core work + 2 hours integration contingency = 12-16 hours planned. Escalate blockers early; no bonus features within the buffer.

DEVELOPER 5 / 10-14 HOURS INCLUDING BUFFER

Research, documentation and submission

Ownership: docs/report; docs/architecture/UML; README; docs/test-results; evaluation tables; presentation and demo checklist. Choose final file formats to match the coursebook.

Work block	Core estimate
Research existing healthcare/data-sharing approaches	2-3 h
Write roles, requirements, data table and UML	2-3 h
Assemble tests, measurements and run instructions	2-3 h
Prepare presentation and independently rehearse	2-3 h

Implement in this order

1. Read the coursebook report limits and Week 6 presentation format on Day 1. They were not included in the pasted manual. Collect two or three relevant, cited examples of current healthcare portals/data-sharing approaches and explain what they do well and where user control is limited.
2. Document this exact scope: one guardian-held child record, fixed clinic, scoped school/doctor access, reward and audit. Include actors, functional requirements and the on-chain/off-chain/hash mapping table.
3. Maintain the component and access-sequence UML from this plan, updating names only when interfaces actually change. Explain why the local file stays off-chain and why token balances do not grant permission.
4. Collect results from Developers 2 and 4. Include the reason each test matters, pass/fail evidence, deployment/per-function gas and local timing/scaling tables. Mark unavailable measurements honestly; never invent results.
5. Independently follow the README on the local lab environment. Capture concise evidence of denied access, guardian grant/reward, scoped outputs, revocation, expiry and tampering. Report any blocker to its code owner.
6. Prepare a short presentation following the lab format and a two-to-three-minute live demonstration. Every person explains their own module. Freeze the report and demo after the final tests pass.

Handoff

Start research/report structure immediately; do not wait for finished code. Receive initial diagrams on Day 2, observed test results on Day 4 and final measurements by Day 6.

Done when

- The report contains every manual deliverable and conforms to coursebook length/structure.
- All research claims have sources, and all test/gas claims have measured evidence.
- A teammate can reproduce the deployment and demo using the instructions alone.
- The presentation explains limitations and clearly distinguishes the prototype from production healthcare software.

Time boundary: 8-12 hours core work + 2 hours integration contingency = 10-14 hours planned. Escalate blockers early; no bonus features within the buffer.

SCHEDULE / SHARED DEPENDENCIES

One week, with a feature freeze

Day	Team outcome	Who leads
1	Agree signatures, scopes, event fields and fixture bytes. Compile/test a lab contract and call it from Python. Confirm coursebook requirements.	All; Dev 4 connection gate
2	Registry/consent/token implementation; JSON/hash helpers; tests and report skeleton proceed in parallel.	Devs 1-3; Dev 5 research
3	Deploy the three contracts; first real registration/grant/access; exchange stable ABIs and helper results.	Devs 1 and 4
4	Full school/doctor workflow with audit and reward. Run negative tests. Freeze features.	Devs 2-4
5	Fix failures; test revocation, exact expiry, tampering and repeated rewards. Independent README check.	All; Dev 2 tests
6	Measure gas/time for 1, 5 and 10 requesters. Finish result tables, report and slides.	Devs 4 and 5
7	Final clean run and presentation rehearsal. Use remaining buffer only for blockers.	All

Agreements to freeze on Day 1

- Owner means the guardian's authenticated wallet, not the deploying administrator. Never let a caller grant for an arbitrary owner address.
- Scope codes, duration units, hash prefixes, function arguments and event reason codes must match across Solidity, Python and tests.
- Developer 4 owns chain.py; Developer 3 owns main.py/records.py/disclosure.py. Agree results before wiring the menu so both can work independently.
- Developer 2 owns test files; Developer 1 fixes registry/consent logic, and Developer 2 fixes token logic. Failed integration is shared work, not automatically Developer 4's entire responsibility.

Review and fallback rules

Developer 2 reviews Developer 1's permission checks. Developer 1 reviews token mint authority. Developer 3 reviews event-to-response handling with Developer 4. Developer 5 independently repeats the complete demo.

If the Day 1 connection check fails, use the lab template and pair Developers 1 and 4 to solve it immediately. Do not start a website while core integration is blocked. Keep the record and actors fixed; cut presentation polish before any mandatory test, logging or reward requirement.

The target is below 20 hours per person, but not a guaranteed cap. If setup requires learning both stacks from scratch, tell the team on Day 1 and seek course guidance rather than silently removing assessed features.

VALIDATION / WHAT TO SHOW EXAMINERS

Required tests and honest measurements

Test group	Cases to cover	Why it matters
Registration / clinic	Register; duplicate/zero hash; wrong signer; missing guardian; attempted hash replacement	Binds evidence to approved identity and issuer
Consent	Valid grant; 0/366 days; wrong requester/scope; exact expiry; revoke; another owner	Prevents unauthorized or stale access
Audit / integrity	Granted event; denied event survives; missing record; changed hash; unavailable RPC	Shows auditable attempts and no disclosure on failure
Rewards	First grant; active duplicate; revoke/regrant; expiry/regrant; unauthorized mint; access unchanged balance	Demonstrates incentive without bypass or repeated rewards
Full workflow	Register -> attest -> deny -> grant/reward -> access -> revoke -> deny; doctor scope; expiry	Shows components actually work together

Unit and integration evidence

Run IdentityRegistry.t.sol, ConsentManager.t.sol and ConsentRewardToken.t.sol using the Solidity-test setup from Lab 3. Integration tests may use the Python workflow, but do not replace the Solidity unit tests. For each result record test ID, requirement, expected outcome, actual outcome and status. Use only synthetic data.

Gas table to produce

Columns: contract/function, scenario, successful sample count, total gasUsed, average gasUsed and notes. Report deployment gas for all three contracts individually. Include registration, vaccination attestation, first rewarded grant, later unrewarded grant, revoke, allowed access and denied access. At least five comparable calls per repeatable scenario is a useful small sample; state the actual count.

Do not mix first-write and overwrite/regrant cases into an unexplained average. The internal reward mint cost is already included in the grant transaction; do not double count it. Read-only RPC queries incur no transaction fee for the caller; describe them separately rather than inventing receipt gas.

Small scaling/timing table

Run the same local scenario with 1, 5 and 10 requester accounts. Record actual request count, mean gas per operation, mean send-to-receipt time, mean send-to-decoded-event time and failures. Use a monotonic clock and document automining, software versions and machine settings. Reset/redeploy explicitly between comparable runs if needed.

Report gas units, not a made-up fiat cost. Local timing is not evidence of public-network performance. A simple optimization is to emit events rather than duplicating an ever-growing access array in storage; measure and explain the chosen design. Do not create complex optimizations solely for a benchmark.

FINISH / COMPLETE THE ASSIGNMENT

Submission checklist and demo script

Manual deliverable	What this team submits
Research and planning	Short cited background, problem statement, roles and functional requirements
System design	Architecture/sequence UML, data mapping table, scope/expiry/revocation and audit design
Implementation	Three Solidity contracts, Python app, synthetic local fixtures and reproducible local deployment
Testing	Named Solidity test files, integration workflow and results with criticality explanations
Evaluation	Actual deployment/per-function gas and small-user-count timing/cost tables
Documentation / presentation	Coursebook-compliant report, clear compile/deploy/test/run instructions and Week 6-format presentation

Two-to-three-minute demonstration

1. Show the synthetic JSON locally and its on-chain commitment; raw medical values are absent from the contract.
2. Attempt school access before consent: show a denied audit event and no health data.
3. Guardian grants school consent: show the first reward and status-only response.
4. Guardian grants doctor scope: show vaccine/date only, with no batch or whole-file download.
5. Revoke school consent and show denied access. Use controlled local test-time advancement to demonstrate expiry without waiting days.
6. Tamper with a copy of the JSON and show integrity denial. Restore the fixture, then show the passing tests and measured gas table.

Deliberately excluded

No website, public testnet, Fabric/Fabric-X, Java services, real patient data, research marketplace, token trading, encryption/key recovery, general record editing, multi-guardian governance or advanced distributed recovery. Frontend work is bonus-only after every required result is complete.

Clarifications to check, not silently assume

The manual prefers Python and makes local Hardhat deployment sufficient. Its wording around "contract owner" and "tokens enable access" is inconsistent with owner-controlled consent and its explicit statement that consent alone controls access. This plan uses guardian-owned consent and reward-only tokens; verify any narrower lab interpretation on Day 1. No token standard is specified in the supplied text.

The Lab 3 starter, coursebook page limits and Week 6 presentation format were not supplied. Reuse/check those materials before final submission. This PDF is a work plan, not the final assessed research report or a claim of guaranteed marks.

Sources and version basis

Primary: the assignment manual pasted by the user, especially Steps 1-7; vaccination_blockchain_concept.pdf for the guardian/local-record scenario. Technical reference: <https://hardhat.org/> (checked 24 September 2026) confirms Solidity-test support. Pin the lab's working versions rather than copying a guessed version number from this plan.